

Library Management System

Introduction to Programming 2 | Aimukhambetov Olzhas

Group Members: Yerdan Orynassarov, Nurtas Amantay, Yerasyl Madiuly, Dauren Tilek

Problem Description

✗ No tracking

Manual records, hard to track who borrowed what

✗ No availability info

No way to know if a book is free or taken

✗ No history

Borrowing history of users is not recorded

Our Goal

Build a Python app that:

- ✓ Manages books & users
- ✓ Tracks borrow/return
- ✓ Shows availability
- ✓ Saves history to JSON

System Design

main.py

Entry point · Menu loop

services/

LibraryService — business logic

models/

Book · User

utils/

Validation

JSON Files

books.json · users.json

OOP Principles Used

- Classes & Objects
- Encapsulation
- Modular design
- Separation of concerns
- File persistence

Demo of the Program

```
Library Management System
```

```
1. Show books
```

```
2. Add book
```

```
3. Borrow book
```

```
4. Return book
```

```
5. Show user history
```

```
6. Show statistics
```

```
7. Exit
```

```
Choose option: 3
```

```
Enter user ID: 2
```

```
Enter book ID: 3
```

```
Book borrowed successfully
```

What the system can do:



Show all available books



Add new book to catalog



Borrow book (marks unavailable)



Return book (marks available)



View user borrow history



Show library statistics

Team Contribution Breakdown

Student 1 - Yerdan

Classes and Data Models

- models/book.py
- models/user.py
- models/person.py

Student 2 - Nurtas

Business Logic

- file_service.py
- library_service.py
- Add/search/borrow books

Student 3 - Yerassyl

Utilities & Testing

- utils/
- tests/
- JSON files

Student 4 - Dauren

Application Integration

- main.py
- Application flow
- README

Discussion of Key Decisions

Complex Parts

Borrow logic: check availability before allowing

Return logic: match user + book ID correctly

JSON sync: read-modify-write on every action

Interesting Solutions

Service layer isolates business logic from UI

JSON as lightweight, human-readable database

Try/except in main() catches all bad inputs

Optimizations

Data loaded once at startup into memory

Search by ID — $O(n)$ scan but fast for small sets

Modular structure: easy to extend later

Conclusion

- ✓ Implemented full OOP architecture with models, services, utils
- ✓ Covered all core library operations: add, borrow, history
- ✓ Used JSON for persistent, human-readable data storage
- ✓ Written unit tests to verify core logic
- ✓ Clear team split: each member owned a module end-to-end